Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа кибербезопасности

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 3

«Реализация моделей безопасности в ОС Windows»

по дисциплине «Модели безопасности компьютерных систем»

Выполнил
студент гр. 5131001/30501 Чупров Е. А.

<подпись>

Проверил
ассистент Жуковский Е. В.

<подпись>

Санкт-Петербург
2026

СОДЕРЖАНИЕ

1 Формулировка задания 3

2 Теоретические сведения 4

3 Результаты работы 7

4 Выводы 11

1 ФОРМУЛИРОВКА ЗАДАНИЯ

Разработать драйвер-фильтр для ОС Windows 7/8/8.1/10/11, позволяющий ограничить права доступа процессов к объектам файловой системы. Разграничение должно осуществляться по правилам Дискреционной модели доступа/Ролевой модели доступа (в зависимости от варианта).

В ходе выполнения лабораторной работы необходимо выполнить следующие действия:

1) Исследовать методы разработки драйверов для ОС Windows, изучить типы драйверов файловой системы, выбрать подходящий тип драйвера.

2) Подготовить среду разработки, которая включает в себя Visual Studio, Window Driver Kit (WDK), Software Development Kit (SDK).

3) Подготовить целевую ОС, на которой будет работать разработанный драйвер, для возможности установки неподписанных драйверов.

4) Изучить существующие проекты драйверов. Выбрать наиболее подходящий для поставленной задачи и скомпилировать его.

5) Разработать простейшую программу, позволяющую читать и записывать в текстовый файл данные. Название файла и данные для записи подаются в качестве аргументов командной строки.

6) Доработать выбранный драйвер до возможности блокировки запросов от процессов к файлам в определенной директории. Доступ процесса к файлу должен определяться моделью контроля доступа и правами, указанными в отдельном текстовом файле (конфигурационный файл). В качестве процесса выступает разработанная в п.5 программа.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Разграничение доступа определяет, какой субъект какую операцию может выполнить над объектом. Субъектом выступает процесс или учётная запись, объектом — файл, каталог или другой ресурс, а операция задаётся правом: прочитать, записать, создать, удалить. Пока правило не зафиксировано, программа, дошедшая до файла, распоряжается им в пределах учётной записи, от имени которой запущена. Модель выносит правило из содержимого файла и требует проверить его раньше, чем операция изменит данные. Обычная форма такой записи — матрица доступа. По одной оси стоят субъекты, по другой объекты, в клетке перечислены разрешённые права. Дискреционная модель заполняет матрицу по отдельным парам: у одного файла у разных субъектов разные клетки, и право, выданное одному субъекту, к остальным само не переходит. Ролевая модель добавляет промежуточный уровень. Права назначаются роли, субъект получает роль, и один набор действует на всю выбранную область, например на все файлы каталога. Мандатная модель исходит из меток конфиденциальности. Метку назначает система, субъект своим решением её не ослабляет, а доступ вычисляется сравнением метки субъекта и метки объекта. Лабораторная работа делает дискреционный вариант. Субъекты — процессы тестовых программ, объекты — отдельные файлы каталога, права — чтение и запись. Матрица лежит в текстовом конфигурационном файле. Ролевая схема того же задания собиралась бы иначе: у роли вроде администратора или пользователя был бы общий набор прав на все файлы директории.

В Windows прикладная программа к диску сама не обращается. Вызовы CreateFile, ReadFile и WriteFile переходят в интерфейс ядра, и диспетчер ввода-вывода оформляет запрос пакетом IRP. Пакет проходит стек устройств тома и доходит до драйвера файловой системы; для тома NTFS это драйвер NTFS. Между программой и этим драйвером ставится фильтр. Прежний фильтр файловой системы встраивался в стек устройств напрямую и сам передавал запрос вниз, повторяя служебную работу файловой системы. Минифильтр регистрируется у диспетчера фильтров FltMgr, сообщает высоту altitude и список

операций, а диспетчер выстраивает цепочку и вызывает зарегистрированные процедуры. Высота задаёт порядок. Фильтр с большей высотой видит запрос раньше фильтра с меньшей. Группа загрузки, в этой работе FSFilter Activity Monitor, относит драйвер к наблюдающим фильтрам, а место внутри группы определяется высотой. На операцию можно поставить процедуру до выполнения и процедуру после. До выполнения запрос ещё не обработан файловой системой: процедура пропускает его дальше или завершает сама и возвращает статус, в том числе STATUS_ACCESS_DENIED. После выполнения процедура видит уже полученный результат. Для модели доступа нужно первое, потому что запрет должен наступить до изменения объекта.

Чтение и запись в матрице собираются из нескольких запросов ядра. Открытие файла — это IRP_MJ_CREATE: в нём передаются желаемый доступ и способ открытия, то есть открыть существующий файл, создать новый или перезаписать. Байты читает IRP_MJ_READ, байты пишет IRP_MJ_WRITE. Если программа открывает файл с перезаписью, прежняя длина обнуляется ещё при обработке открытия, и отдельный запрос записи для уничтожения содержимого уже не требуется. Право «запись запрещена» поэтому проверяется и на IRP_MJ_WRITE, и на том открытии, которое само меняет файл. Имя объекта в запросе обычно приходит как путь устройства тома, \Device\HarddiskVolume<номер>\каталог\файл. В конфигурации тот же файл записан буквой диска. Буква и номер тома обозначают путь по-разному, и перед сравнением с матрицей фильтр приводит имя к виду C:\..., поскольку защищаемые файлы работы лежат на этом диске. Субъект запроса — процесс того потока, от которого пришёл IRP. Фильтр берёт имя исполняемого файла процесса. Так формулировка задания, где субъект назван пользователем и представлен тестовой программой, совпадает с тем, что видно в момент операции. Дальше имя процесса и путь файла сравниваются с клеткой матрицы. Единица пропускает операцию, ноль завершает её отказом в доступе. Пара, для которой клетки нет, в работе не ограничивается: фильтр применяет только те правила, которые записаны явно.

Рядом с таким фильтром в Windows уже есть собственная дискреционная проверка, и фильтр её не подменяет. У файла NTFS есть дескриптор безопасности: владелец и список управления доступом. Записи списка ссылаются на идентификаторы пользователей и групп и задают маску прав. Проверка выполняется при открытии, успешный результат закрепляется за описателем. Минифильтр этой работы список не читает и не переписывает. Он добавляет решение по текстовой матрице для тех операций, на которые зарегистрирован. Удаление, переименование и отображение файла в память идут другими запросами; пока на них нет процедуры фильтра, матрица их не описывает. Имя исполняемого файла как субъект уже, чем учётная запись: решает имя процесса, а не пользователь, от которого процесс запущен. Код на этом уровне стека подчиняется ещё и правилу загрузки. На 64-разрядной Windows драйвер режима ядра проходит проверку подписи. Тестовый режим загрузки доверяет сертификату, выпущенному для разработки, драйвер подписывается этим сертификатом, а проверка подписи при старте системы остаётся включённой. В результате минифильтр оказывается слоем, где текстовая матрица дискреционного доступа становится пропуском или отказом конкретного запроса к файлу.

3 РЕЗУЛЬТАТЫ РАБОТЫ

Работа выполнена на одной машине под Windows 11 сборки 28000. Для сборки поставлены Visual Studio Community 2026, Windows Driver Kit и Windows SDK версии 10.0.28000, а в состав Visual Studio добавлен набор инструментов WindowsKernelModeDriver10.0. Secure Boot на машине выключен. В конфигурации загрузки включён тестовый режим подписи, после перезагрузки ядро принимает локальный тестовый сертификат. Драйвер подписан этим сертификатом по алгоритму SHA256. Проверка signtool verify /pa завершается успешно. Обход проверки подписи при старте системы не использовался.

Для ограничения доступа выбран минифильтр файловой системы. Такой драйвер подключается к Filter Manager и наблюдает операции над файлами, не подменяя драйвер файловой системы тома. Вариант модели — дискреционный. В текстовом конфигурационном файле для каждого защищаемого пути записаны имена процессов и два признака: чтение и запись. Единица разрешает операцию, ноль запрещает. Если пары «файл и процесс» в файле нет, фильтр её не ограничивает. Тем самым права задаются отдельно для каждого файла, а не одним набором на весь каталог, как было бы в ролевой модели.

Конфигурация читается из C:\DriverConfig\Config.txt. Процессу filetool.exe разрешено читать C:\DriverConfig\data\note.txt и запрещено в него писать. Тот же процесс может читать и писать C:\DriverConfig\data\report.txt. Процессу notepad.exe запрещены и чтение, и запись note.txt. Файл C:\DriverConfig\data\free.txt в конфигурации не упомянут и служит проверкой правила «нет записи — нет запрета».

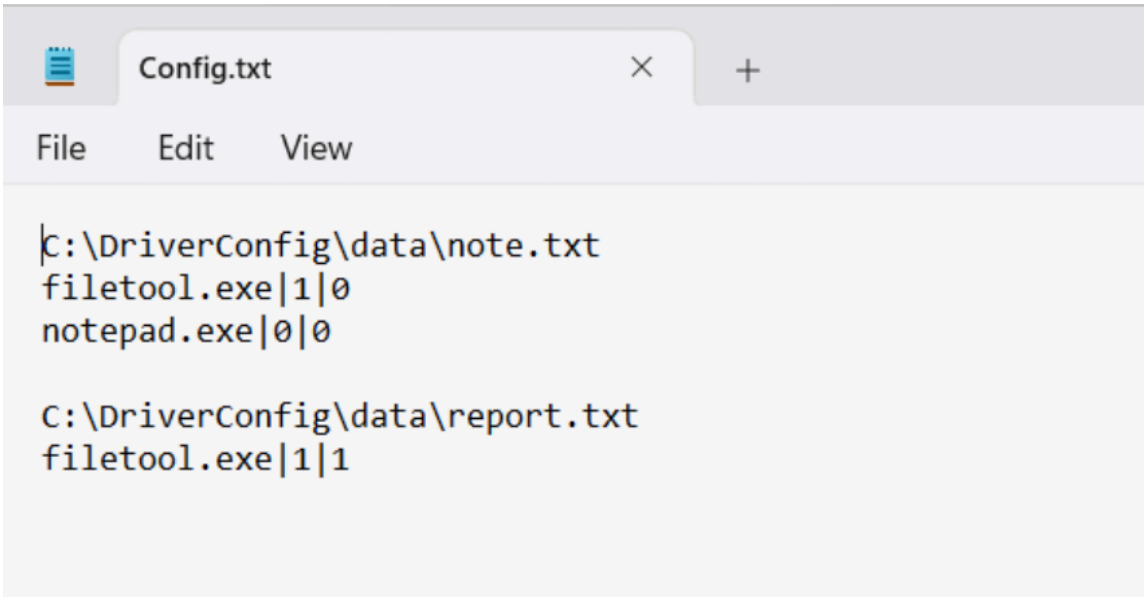


Рисунок 1 – Конфигурация дискреционного доступа

Проверка выполняется консольной программой filetool. Ей передаётся команда r или w, путь к файлу и, при записи, текст. Драйвер различает процессы по имени исполняемого файла, поэтому одна и та же программа, скопированная под именем notepad.exe, попадает под другие права.

Минифильтр зарегистрирован на операции чтения, записи и открытия файла. На каждом таком обращении он заново читает конфигурацию, сравнивает имя файла и имя процесса без учёта регистра и при запрете завершает операцию со статусом STATUS_ACCESS_DENIED. Путь вида \Device\HarddiskVolume<номер>\... приводится к виду C:\.... Имя процесса берётся из образа и освобождается после проверки. Открытие обрабатывается отдельно: программа записи создаёт файл с усечением ещё до запроса записи. Если запретить только запись, файл успевает очиститься, хотя новые байты в него не попадают. Запрет открытия не даёт усечению обойти правило.

Служба FsFilterX создана как драйвер файловой системы с зависимостью от FltMgr, группой загрузки FSFilter Activity Monitor и высотой 370030. Раздел установки в INF рассчитан на сборки до 25952, а установленная система имеет сборку 28000, поэтому фильтр загружен командой fltmc load, а не установщиком INF. В списке Filter Manager фильтр стоит на высоте 370030 и подключён к тому C:.

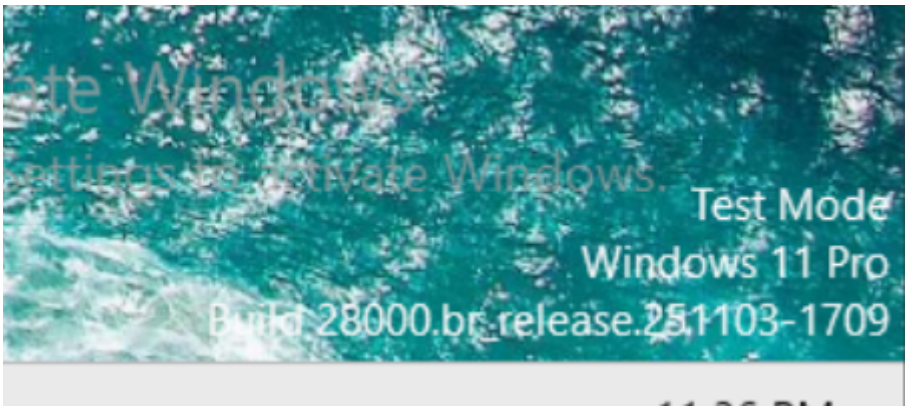


Рисунок 2 – Тестовый режим подписи драйверов

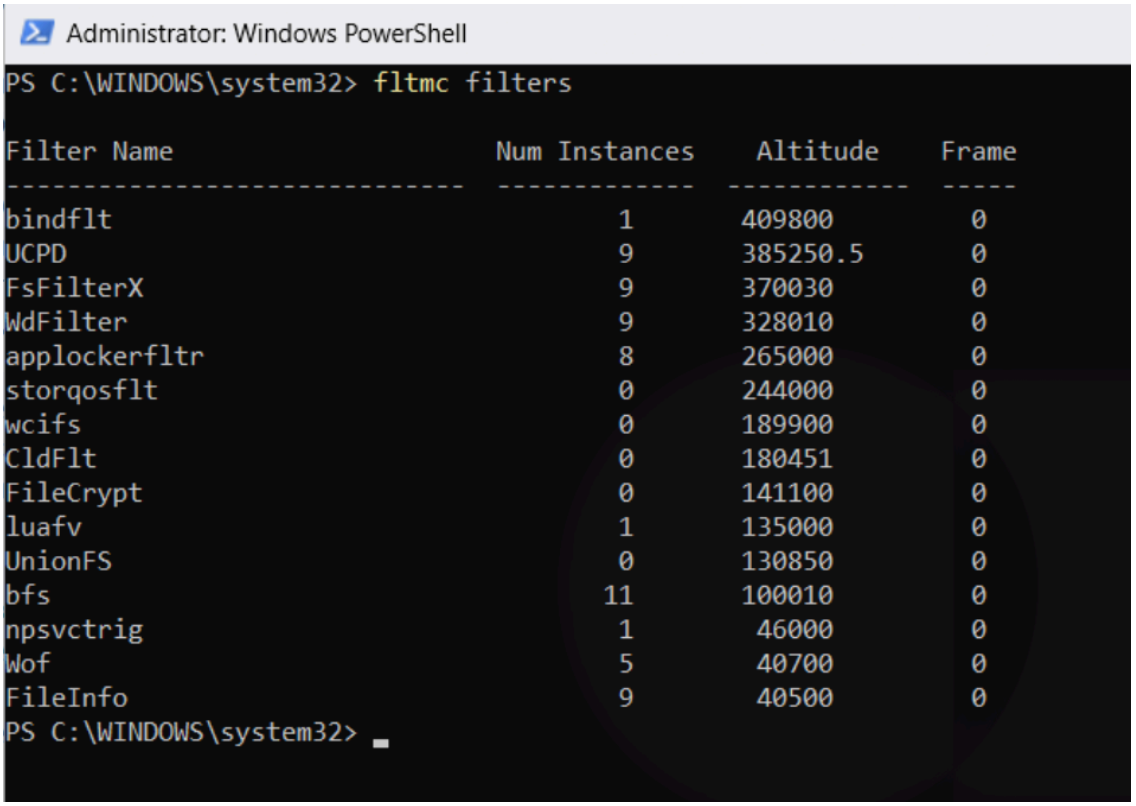


Рисунок 3 – Загруженный минифильтр FsFilterX

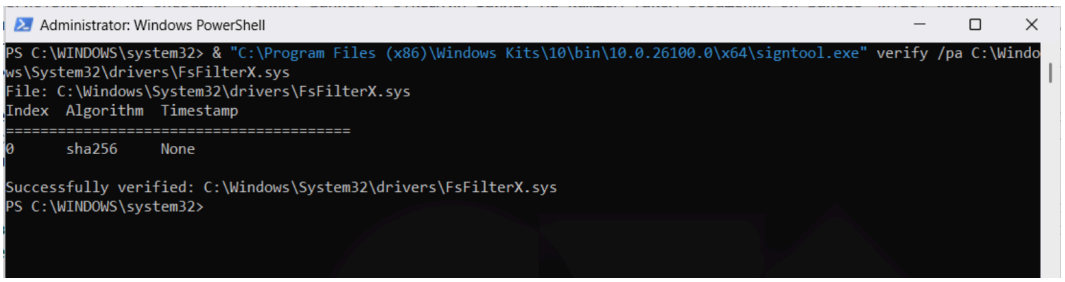


Рисунок 4 – Проверка тестовой подписи драйвера

После загрузки фильтра filetool.exe читает note.txt и получает сохранённый текст hello. Запись строки hacked в тот же файл отклоняется: программа печатает file openning failed, а повторное чтение снова показывает hello. Запись и чтение report.txt проходят, в файле оказывается переданный текст. Запись в free.txt, которого нет в конфигурации, также проходит.

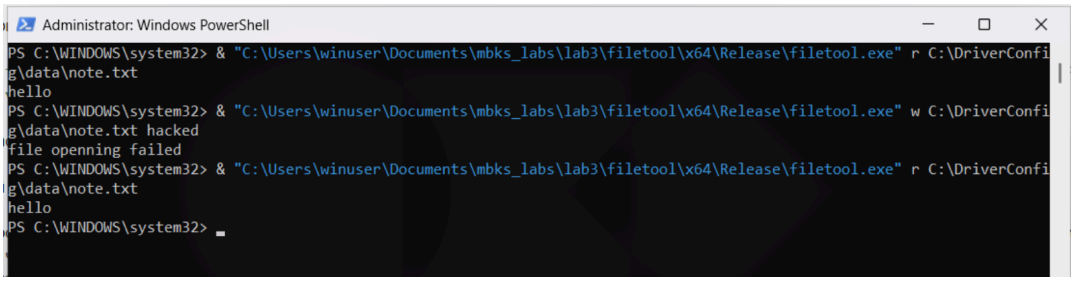


Рисунок 5 – Чтение note.txt разрешено, запись запрещена

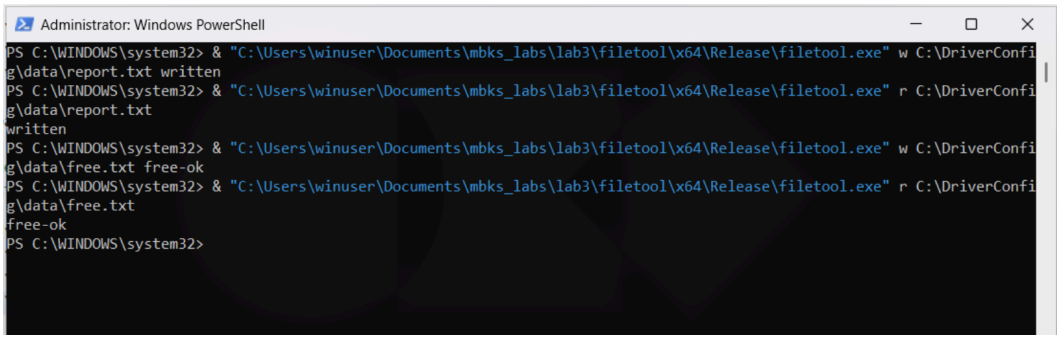


Рисунок 6 – Запись report.txt разрешена, файл вне конфигурации не ограничивается

Копия filetool.exe, запущенная как notepad.exe, не может ни прочитать, ни записать note.txt. Оба вызова печатают file openning failed. Последующее чтение тем же filetool.exe по-прежнему показывает hello, то есть запрещённое открытие файл не изменило.

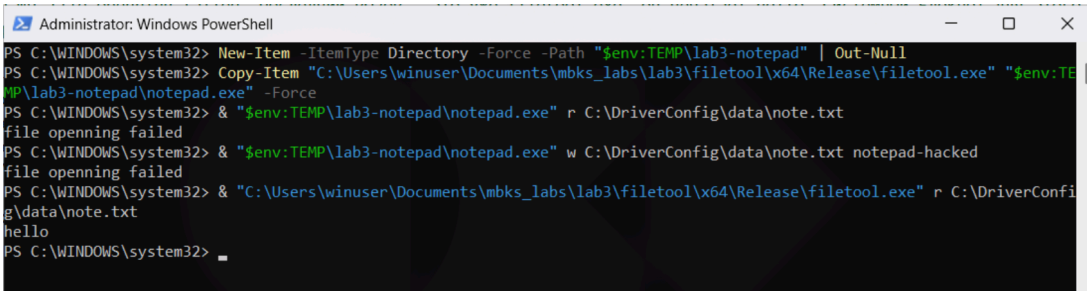


Рисунок 7 – Процесс notepad.exe не читает и не записывает note.txt

4 ВЫВОДЫ

В работе на уровне ядра Windows реализована дискреционная модель доступа к файлам выбранного каталога. Права хранятся в отдельном текстовом файле: у каждого указанного процесса свои разрешения на чтение и запись конкретного файла. Минифильтр применяет эти правила к операциям открытия, чтения и записи и отклоняет запрещённую операцию, не уничтожая уже записанные данные. Проверка на filetool.exe и на копии с именем notepad.exe показывает, что решение зависит от пары «процесс — файл», а файл, не внесённый в конфигурацию, остаётся доступным.

Фильтр не заменяет списки доступа NTFS и не описывает отдельные права на удаление и переименование. Его область — чтение и запись тех файлов, которые перечислены в конфигурации.